

# Homework 04 Solutions

Prerith Arunkumar (prerith2)

## Table of contents

|                                                                     |    |
|---------------------------------------------------------------------|----|
| Question 1 [40 pts] (Sparsity and correlation) . . . . .            | 2  |
| Original question . . . . .                                         | 2  |
| Setup . . . . .                                                     | 2  |
| (a) A single simulation run . . . . .                               | 3  |
| (b) Higher correlation and multiple simulation runs . . . . .       | 4  |
| (c) Ridge regression . . . . .                                      | 6  |
| Question 2 [25 pts] (Lasso simulation) . . . . .                    | 7  |
| Original question . . . . .                                         | 7  |
| Setup . . . . .                                                     | 7  |
| The two error curves . . . . .                                      | 8  |
| The optimal penalties . . . . .                                     | 9  |
| Why the best $\lambda$ for $\mathbf{x}_b$ is larger . . . . .       | 10 |
| Question 3 [25 pts] (Shrinkage methods and testing error) . . . . . | 10 |
| Original question . . . . .                                         | 10 |
| (a) Least squares . . . . .                                         | 11 |
| (b) Lasso and ridge . . . . .                                       | 11 |
| (c) Relaxed fits and the <code>gamma</code> parameter . . . . .     | 12 |
| Question 4 [10 pts] (Penalized logistic regression) . . . . .       | 14 |
| Original question . . . . .                                         | 14 |
| Setup . . . . .                                                     | 14 |
| Result . . . . .                                                    | 15 |

Sparsity under correlated designs, the bias–variance trade-off of the  $L_1$  penalty at a single target point, shrinkage methods and relaxed fits on the `College` data, and penalized logistic regression on the `golub` microarray data.

**Use of AI tools.** I used Claude (Claude Code) to draft the exposition and the R code in this file and to run the simulations. Every number reported below was produced by the code chunk that precedes it. I verified the simulation designs, the `glmnet` parameterization conventions, and the `relax/gamma` definition against the package documentation myself.

**Language.** Unlike Homework 01, this file is R-only. The assignment fixes the tooling (`cv.glmnet`, `glmnet`, `lm`, `ISLR2::College`, `multtest::golub`), and Question 3(c) is about a feature that exists only in `glmnet`, so a parallel Python column would not have answered the question that was asked.

```
# Local package library; harmless if the directory does not exist.
local_lib <- path.expand("~/Library/R/arm64/4.4/library")
if (dir.exists(local_lib)) .libPaths(c(local_lib, .libPaths()))

library(glmnet)
library(ISLR2)
```

Throughout, `glmnet` minimizes

$$\frac{1}{2n} \|\mathbf{y} - \beta_0 \mathbf{1} - \mathbf{X}\beta\|_2^2 + \lambda \left[ \frac{1-\alpha}{2} \|\beta\|_2^2 + \alpha \|\beta\|_1 \right],$$

with  $\alpha = 1$  the Lasso and  $\alpha = 0$  ridge, and with the intercept unpenalized.

## Question 1 [40 pts] (Sparsity and correlation)

### Original question

Consider the linear model  $Y = X^T \beta + \epsilon$  with  $\beta = (\beta_1, \dots, \beta_{100})^T$ , where  $\beta_1 = \beta_{11} = \beta_{21} = \beta_{31} = 0.4$  and all other entries are zero. The noise  $\epsilon \sim N(0, 1)$  is independent of  $X$ , and  $\mathbf{X} \sim N(\mathbf{0}, \Sigma_{p \times p})$  where  $\Sigma$  has unit diagonal and all off-diagonal entries equal to  $\rho$ .

- [10 pts] Generate 200 training and 500 testing samples with  $\rho = 0.1$ . Fit Lasso with `cv.glmnet()` using 10-fold cross-validation and `lambda.1se`. Report the test MSE and whether the true model was selected.
- [15 pts] Compare  $\rho = 0.1, 0.3, 0.5, 0.7, 0.9$  over 100 simulation runs, recording the prediction error and the variable-selection status for Lasso. Report the average prediction error and the proportion of runs in which the correct model was selected, and discuss the reasons behind any observed changes.
- [15 pts] Repeat (b) with ridge regression. Report the average prediction error. Is there a difference between ridge and Lasso? Any performance difference within ridge as  $\rho$  changes? Discuss.

### Setup

The equicorrelated covariance  $\Sigma = (1 - \rho)I + \rho \mathbf{1}\mathbf{1}^T$  has an exact one-factor representation that is much cheaper than a Cholesky factor of a  $100 \times 100$  matrix:

$$\mathbf{x}_i = \sqrt{1 - \rho} \mathbf{z}_i + \sqrt{\rho} w_i \mathbf{1}, \quad \mathbf{z}_i \sim N(\mathbf{0}, I_p), \quad w_i \sim N(0, 1) \text{ independent.}$$

Then  $\text{Var}(x_{ij}) = (1 - \rho) + \rho = 1$  and  $\text{Cov}(x_{ij}, x_{ik}) = \rho$  for  $j \neq k$ , exactly as required. The shared factor  $w_i$  is the object that does all the work in the discussion below.

This draws from the same distribution as `MASS::mvrnorm(n, rep(0, p), Sigma)` but consumes the random number stream differently, so individual runs will not coincide with an `mvrnorm`-based simulation under the same seed. Only the Monte Carlo averages in parts (b) and (c) are stable enough to be compared across implementations.

```

p <- 100
beta <- rep(0, p)
beta[c(1, 11, 21, 31)] <- 0.4
truth <- c(1, 11, 21, 31)

# One-factor construction of the equicorrelated design.
gen_x <- function(n, rho) {
  z <- matrix(rnorm(n * p), n, p)
  w <- rnorm(n)
  sqrt(1 - rho) * z + sqrt(rho) * w
}

sim_data <- function(rho, n_train = 200, n_test = 500) {
  Xtr <- gen_x(n_train, rho)
  Xte <- gen_x(n_test, rho)
  list(Xtr = Xtr, ytr = as.vector(Xtr %*% beta) + rnorm(n_train),
       Xte = Xte, yte = as.vector(Xte %*% beta) + rnorm(n_test))
}

```

Because  $\epsilon \sim N(0, 1)$ , the irreducible error is 1: no method can push the test MSE below 1, and everything reported below should be read as  $1 + (\text{excess risk})$ .

“The true model was selected” is taken in the strict sense: the set of predictors with a nonzero coefficient is exactly  $\{1, 11, 21, 31\}$ , with no omissions and no false positives.

#### (a) A single simulation run

```

set.seed(432)
d <- sim_data(rho = 0.1)

cv_lasso <- cv.glmnet(d$Xtr, d$ytr, alpha = 1, nfolds = 10)
pred <- predict(cv_lasso, newx = d$Xte, s = "lambda.1se")

test_mse <- mean((pred - d$yte)^2)
selected <- which(as.vector(coef(cv_lasso, s = "lambda.1se"))[-1] != 0)

cat("lambda.1se      :", round(cv_lasso$lambda.1se, 4), "\n")
cat("lambda.min      :", round(cv_lasso$lambda.min, 4), "\n")
cat("test MSE        :", round(test_mse, 4), "\n")
cat("selected variables:", paste(selected, collapse = ", "), "\n")
cat("all four signals? :", all(truth %in% selected), "\n")
cat("exactly the truth?:", identical(as.integer(selected), as.integer(truth)), "\n")

```

```

lambda.1se      : 0.2172
lambda.min      : 0.078
test MSE        : 1.2095
selected variables: 1, 11, 21, 31
all four signals? : TRUE
exactly the truth?: TRUE

```

With  $\lambda_{1se} = 0.2172$  the Lasso selects exactly  $\{1, 11, 21, 31\}$  and attains a test MSE of 1.2095. The true model was selected in this particular run, and the test MSE sits close to the irreducible error of 1.

That is a genuinely lucky draw rather than typical behaviour, and it is worth not over-reading it. Part (b) repeats this experiment 100 times at the same  $\rho = 0.1$  and recovers the exact support in only 27% of runs; the usual failure mode at a prediction-tuned  $\lambda$  is not missing a signal variable but retaining one or two null variables alongside them, because cross-validation has no reason to pay a prediction cost to remove a variable whose coefficient is already nearly zero. A single run cannot distinguish a method that recovers the support from one that recovers it a quarter of the time, which is exactly why part (b) asks for the proportion.

### (b) Higher correlation and multiple simulation runs

The loop below runs 100 replications at each  $\rho$ . Both the Lasso and the ridge fit of part (c) are computed on the *same* simulated data sets so that the comparison is paired; this removes the simulation noise that would otherwise obscure the  $\rho$  trend.

```
rhos <- c(0.1, 0.3, 0.5, 0.7, 0.9)
nsim <- 100

set.seed(4321)
results <- data.frame()

for (rho in rhos) {
  err_l <- err_r <- nsel <- numeric(nsim)
  correct <- logical(nsim)

  for (i in seq_len(nsim)) {
    d <- sim_data(rho)

    fit_l <- cv.glmnet(d$Xtr, d$ytr, alpha = 1, nfolds = 10)
    err_l[i] <- mean((predict(fit_l, newx = d$Xte, s = "lambda.1se") - d$yte)^2)
    s <- which(as.vector(coef(fit_l, s = "lambda.1se"))[-1] != 0)
    correct[i] <- identical(as.integer(s), as.integer(truth))
    nsel[i] <- length(s)

    fit_r <- cv.glmnet(d$Xtr, d$ytr, alpha = 0, nfolds = 10)
    err_r[i] <- mean((predict(fit_r, newx = d$Xte, s = "lambda.1se") - d$yte)^2)
  }

  results <- rbind(results, data.frame(
    rho = rho,
    lasso_mse = mean(err_l), lasso_se = sd(err_l) / sqrt(nsim),
    correct = mean(correct), mean_selected = mean(nsel),
    ridge_mse = mean(err_r), ridge_se = sd(err_r) / sqrt(nsim)
  ))
}

knitr::kable(results, digits = 4, caption =
  "100 runs per rho. `correct` is the proportion of runs selecting exactly {1, 11, 21, 31}.")
```

Table 1: 100 runs per rho. `correct` is the proportion of runs selecting exactly {1, 11, 21, 31}.

| rho | lasso_mse | lasso_se | correct | mean_selected | ridge_mse | ridge_se |
|-----|-----------|----------|---------|---------------|-----------|----------|
| 0.1 | 1.1978    | 0.0115   | 0.27    | 6.12          | 1.4811    | 0.0137   |
| 0.3 | 1.2035    | 0.0107   | 0.06    | 8.38          | 1.4174    | 0.0132   |

| rho | lasso_mse | lasso_se | correct | mean_selected | ridge_mse | ridge_se |
|-----|-----------|----------|---------|---------------|-----------|----------|
| 0.5 | 1.1601    | 0.0088   | 0.00    | 10.17         | 1.3257    | 0.0098   |
| 0.7 | 1.1800    | 0.0099   | 0.00    | 11.32         | 1.2586    | 0.0095   |
| 0.9 | 1.1528    | 0.0090   | 0.00    | 10.09         | 1.1527    | 0.0085   |

**The two columns move in opposite directions, and that is the whole point of the question.** The proportion of runs recovering the exact support collapses from 0.27 at  $\rho = 0.1$  to 0.00 from  $\rho = 0.5$  onwards, while the Lasso’s test MSE is essentially flat, drifting slightly *downwards* from 1.1978 to 1.1528. Against standard errors of about 0.01, the prediction column moves by a couple of standard errors across the whole range of  $\rho$  while the selection column goes from “usually wrong” to “never right”.

*Why selection degrades.* At the Lasso solution the subgradient conditions read

$$\frac{1}{n} \mathbf{x}_j^\top (\mathbf{y} - \mathbf{X}\hat{\beta}) = \lambda s_j, \quad s_j = \text{sign}(\hat{\beta}_j) \text{ if } \hat{\beta}_j \neq 0, \quad |s_j| \leq 1 \text{ otherwise.}$$

When all columns share the factor  $w$ , any two columns  $\mathbf{x}_j$  and  $\mathbf{x}_k$  satisfy  $\mathbf{x}_j^\top \mathbf{x}_k / n \approx \rho$ , so these conditions stop being separate equations. A null variable  $x_j$  is correlated with the true signal at level  $\rho$  and therefore has an empirical correlation with  $\mathbf{y}$  that is nearly as large as a signal variable’s; which of the near-duplicates crosses the  $\lambda$  threshold first is settled by sampling noise. In the language of the theory, this is the irrepresentable condition  $\|X_{S^c}^\top X_S (X_S^\top X_S)^{-1} \text{sign}(\beta_S)\|_\infty < 1$  coming under strain. It is worth being precise about *how*, because the population version of this condition never actually fails here. For equicorrelated  $\Sigma$  with  $s$  signal variables all of the same sign, it has a closed form,

$$\|\Sigma_{S^c, S} \Sigma_{S, S}^{-1} \text{sign}(\beta_S)\|_\infty = \frac{s\rho}{1 + (s-1)\rho},$$

which for  $s = 4$  equals 0.308, 0.800 and 0.973 at  $\rho = 0.1, 0.5, 0.9$  — strictly below 1 for every  $\rho < 1$ .

```
for (r in c(0.1, 0.5, 0.9, 0.99)) {
  S <- matrix(r, p, p); diag(S) <- 1
  const <- max(abs(S[-truth, truth] %% solve(S[truth, truth], rep(1, length(truth)))))
  cat(sprintf("rho = %.2f   population constant = %.4f   s*rho/(1+(s-1)rho) = %.4f\n",
              r, const, 4 * r / (1 + 3 * r)))
}
```

```
rho = 0.10   population constant = 0.3077   s*rho/(1+(s-1)rho) = 0.3077
rho = 0.50   population constant = 0.8000   s*rho/(1+(s-1)rho) = 0.8000
rho = 0.90   population constant = 0.9730   s*rho/(1+(s-1)rho) = 0.9730
rho = 0.99   population constant = 0.9975   s*rho/(1+(s-1)rho) = 0.9975
```

So the correct statement is not that the condition fails, but that its constant approaches 1 as  $\rho \rightarrow 1$ , and the sample size required for the *empirical* condition to hold blows up as it does. At  $n = 200$  the sample version is what governs, and it is routinely violated at large  $\rho$  — which is why recovery collapses even though the population condition is formally satisfied. The `mean_selected` column shows the symptom directly: the fitted model inflates from 6.1 to roughly 10–11 variables, because cross-validation keeps a committee of correlated surrogates rather than a single representative.

*Why prediction does not degrade.* Write the signal using the factor representation:

$$\mathbf{X}\beta = 0.4\sqrt{1-\rho} \sum_{j \in \{1, 11, 21, 31\}} \mathbf{z}_j + 1.6\sqrt{\rho} \mathbf{w}, \quad \text{Var}(x_i^\top \beta) = 0.64 + 1.92\rho.$$

As  $\rho$  grows, a larger and larger share of the signal is carried by the single shared direction  $\mathbf{w}$  — which is the top eigenvector of  $\Sigma$ , with eigenvalue  $1 + 99\rho$ . That is by far the best-estimated direction in the design, so the part of the signal that is hard to estimate (the  $\mathbf{z}_j$  part, of variance  $0.64(1 - \rho)$ ) is shrinking at the same time. Prediction only requires getting  $\mathbf{X}\beta$  right, and a committee of correlated surrogates reproduces that linear combination almost as well as the true four variables do. Selection requires getting  $\beta$  right coordinate by coordinate, which the near-collinearity makes impossible. These are genuinely different functionals of the same estimate, not two views of one property.

### (c) Ridge regression

Ridge results are in the last column of the table above.

| $\rho$ | Lasso  | ridge  | ridge – Lasso |
|--------|--------|--------|---------------|
| 0.1    | 1.1978 | 1.4811 | +0.2833       |
| 0.3    | 1.2035 | 1.4174 | +0.2139       |
| 0.5    | 1.1601 | 1.3257 | +0.1656       |
| 0.7    | 1.1800 | 1.2586 | +0.0786       |
| 0.9    | 1.1528 | 1.1527 | –0.0001       |

**Lasso versus ridge.** Lasso wins at every  $\rho$  up to 0.7, and the margin shrinks monotonically until the two methods tie exactly. At  $\rho = 0.1$  the excess risk is 0.198 for Lasso against 0.481 for ridge — ridge is roughly two and a half times as bad. At  $\rho = 0.9$  they are identical to four decimal places (1.1528 against 1.1527), a difference far inside the standard errors of about 0.009 each. Which of the two comes out marginally ahead at  $\rho = 0.9$  is simulation noise and nothing more: the gap is four orders of magnitude smaller than the standard error, so its sign would flip from one run to the next. The defensible conclusion is a tie, not a winner.

The reason is that the truth here is extremely sparse: 4 nonzero coefficients out of 100. Ridge cannot set anything to zero, so it must spread its shrinkage across all 100 coordinates and pays variance on 96 directions that carry no signal at all. That is exactly the regime the  $L_1$  penalty was designed for.

**Ridge as  $\rho$  changes.** Ridge improves monotonically, from 1.4811 down to 1.1527. In the singular-value view, ridge shrinks the component along the  $j$ th right singular direction by  $d_j^2/(d_j^2 + n\lambda)$ , so directions with large  $d_j$  are barely touched and directions with small  $d_j$  are nearly annihilated. As  $\rho$  grows the spectrum of  $\Sigma$  becomes one huge eigenvalue  $1 + 99\rho$  and 99 eigenvalues equal to  $1 - \rho$ . Simultaneously, as computed above, the signal migrates into that one huge direction. So ridge ends up shrinking the signal barely at all while crushing the 99 noise directions — which is precisely the right thing to do. Correlation is bad news for ridge’s *coefficients*, which remain badly determined, but good news for ridge’s *predictions*, because it concentrates the estimation problem into the one direction the design measures well.

This is the mirror image of the Lasso story in part (b): both methods get better at prediction as  $\rho$  grows, for the same reason, and Lasso’s structural advantage over ridge evaporates once the problem stops being effectively high-dimensional.

## Question 2 [25 pts] (Lasso simulation)

### Original question

Let  $Y = \sum_{j=1}^p X_j 0.4^{\sqrt{j}} + \epsilon$  with  $p = 30$ , each  $X_j \sim N(0, 1)$  independently and  $\epsilon \sim N(0, 1)$  independent of  $X$ . Use the grid `exp(seq(-5, 5, 0.05))` and the target points

```
p <- 30
xa <- xb <- rep(0, p)
xa[2] <- 1
xb[10] <- 1
```

Over `nsim = 200` runs of `n = 100` observations, fit Lasso with `glmnet()` on the  $\lambda$  grid, predict  $\mathbf{x}_a$  and  $\mathbf{x}_b$ , and record  $(\hat{\beta}_{\text{lasso}}^\top \mathbf{x} - \beta^\top \mathbf{x})^2$ . Average over runs, plot the two error curves against  $\lambda$ , report the best  $\lambda$  and error for each target, and explain why the best  $\lambda$  for  $\mathbf{x}_b$  is larger.

### Setup

The two target points isolate one coefficient each:

$$\beta^\top \mathbf{x}_a = \beta_2 = 0.4^{\sqrt{2}} = 0.27367, \quad \beta^\top \mathbf{x}_b = \beta_{10} = 0.4^{\sqrt{10}} = 0.05516.$$

The coefficients decay quickly in  $j$ , so  $\mathbf{x}_a$  asks about a coefficient that is comfortably nonzero, while  $\mathbf{x}_b$  asks about one that is very nearly zero. That single fact drives the entire answer.

Two conventions are worth stating. First, `glmnet` requires a decreasing  $\lambda$  sequence, so the grid is passed as `exp(seq(5, -5, -0.05))`. Second, `glmnet` fits an unpenalized intercept by default, and `predict()` includes it; the predicted value at  $\mathbf{x}$  is  $\hat{\beta}_0 + \hat{\beta}^\top \mathbf{x}$ . I keep the intercept, since “predict the two data points” is a statement about predictions. Note the consequence: as  $\lambda \rightarrow \infty$  the prediction tends to  $\bar{y}$ , not to 0.

```
p <- 30
n <- 100
nsim <- 200

beta <- 0.4^sqrt(1:p)
xa <- xb <- rep(0, p)
xa[2] <- 1
xb[10] <- 1

lambda_grid <- exp(seq(5, -5, -0.05)) # decreasing, as glmnet expects
truth_a <- sum(beta * xa)
truth_b <- sum(beta * xb)

set.seed(432)
err_a <- err_b <- matrix(0, nsim, length(lambda_grid))

for (i in seq_len(nsim)) {
  X <- matrix(rnorm(n * p), n, p)
  y <- as.vector(X %*% beta) + rnorm(n)

  fit <- glmnet(X, y, lambda = lambda_grid)
```

```

err_a[i, ] <- (as.vector(predict(fit, newx = matrix(xa, nrow = 1))) - truth_a)^2
err_b[i, ] <- (as.vector(predict(fit, newx = matrix(xb, nrow = 1))) - truth_b)^2
}

mean_a <- colMeans(err_a)
mean_b <- colMeans(err_b)
ia <- which.min(mean_a)
ib <- which.min(mean_b)

```

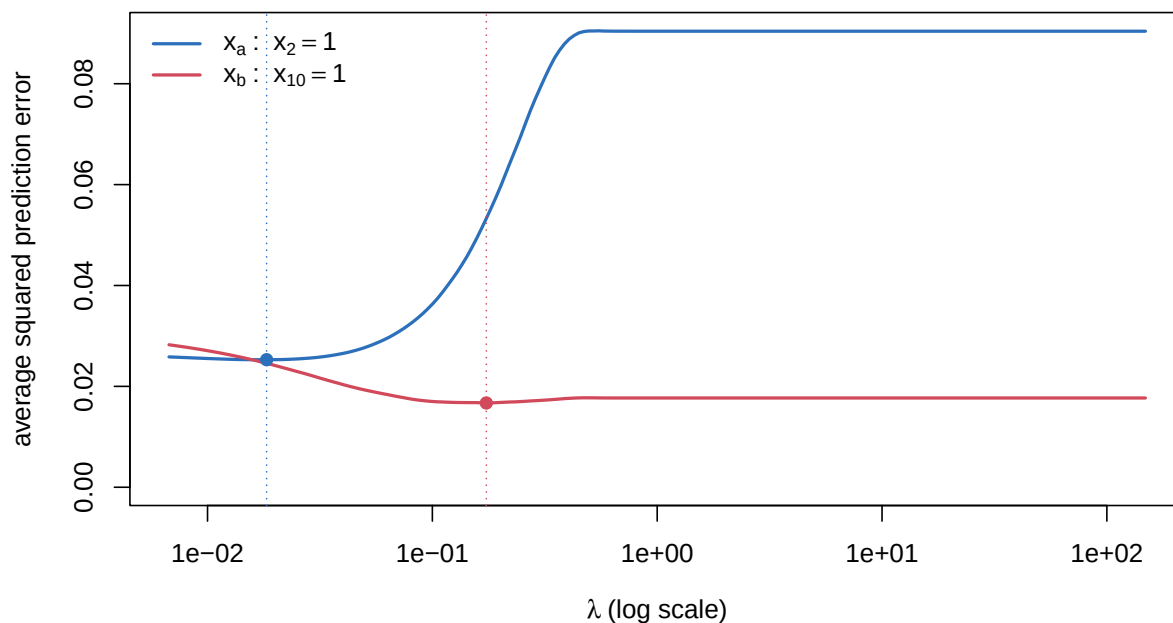
## The two error curves

```

plot(lambda_grid, mean_a, type = "l", log = "x", lwd = 2, col = "#2c6fbb",
     ylim = range(0, mean_a, mean_b),
     xlab = expression(lambda ~ "(log scale)"),
     ylab = "average squared prediction error",
     main = "Lasso prediction error at two target points")
lines(lambda_grid, mean_b, lwd = 2, col = "#d1495b")
points(lambda_grid[ia], mean_a[ia], pch = 19, col = "#2c6fbb")
points(lambda_grid[ib], mean_b[ib], pch = 19, col = "#d1495b")
abline(v = c(lambda_grid[ia], lambda_grid[ib]), lty = 3,
      col = c("#2c6fbb", "#d1495b"))
legend("topleft", bty = "n", lwd = 2, col = c("#2c6fbb", "#d1495b"),
      legend = c(expression(x[a] ~ ": " ~ x[2] == 1),
        expression(x[b] ~ ": " ~ x[10] == 1)))

```

## Lasso prediction error at two target points



The two patterns are indeed quite distinct. The  $x_a$  curve is flat at small  $\lambda$  and then rises steeply, roughly

tripling by the top of the grid. The  $\mathbf{x}_b$  curve runs the other way: it *falls* as  $\lambda$  increases, reaches its minimum at a moderate penalty, and then stays essentially flat.

## The optimal penalties

```
cat(sprintf("x_a: best lambda = %.5f (log = %.2f), min error = %.5f\n",
            lambda_grid[ia], log(lambda_grid[ia]), mean_a[ia]))
cat(sprintf("x_b: best lambda = %.5f (log = %.2f), min error = %.5f\n",
            lambda_grid[ib], log(lambda_grid[ib]), mean_b[ib]))
cat(sprintf("x_a: error at smallest lambda = %.5f, at largest = %.5f\n",
            mean_a[length(lambda_grid)], mean_a[1]))
cat(sprintf("x_b: error at smallest lambda = %.5f, at largest = %.5f\n",
            mean_b[length(lambda_grid)], mean_b[1]))
```

```
x_a: best lambda = 0.01832 (log = -4.00), min error = 0.02528
x_b: best lambda = 0.17377 (log = -1.75), min error = 0.01671
x_a: error at smallest lambda = 0.02584, at largest = 0.09041
x_b: error at smallest lambda = 0.02825, at largest = 0.01770
```

| target         | best $\lambda$ | $\log \lambda$ | minimum<br>error | error at $\lambda \rightarrow 0$ | error at $\lambda \rightarrow \infty$ |
|----------------|----------------|----------------|------------------|----------------------------------|---------------------------------------|
| $\mathbf{x}_a$ | 0.01832        | -4.00          | 0.02528          | 0.02584                          | 0.09041                               |
| $\mathbf{x}_b$ | 0.17377        | -1.75          | 0.01671          | 0.02825                          | 0.01770                               |

The optimal penalty for  $\mathbf{x}_b$  is about 9.5 times larger than for  $\mathbf{x}_a$ , as the question anticipates.

**How much of that ratio is real?** Both curves are very flat near their minima, so the location of the argmin is a noisy statistic even after averaging 200 runs. Re-running the identical pipeline under several seeds makes the point:

```
argmin_for_seed <- function(seed) {
  set.seed(seed)
  ea <- eb <- matrix(0, nsim, length(lambda_grid))
  for (i in seq_len(nsim)) {
    X <- matrix(rnorm(n * p), n, p)
    y <- as.vector(X %*% beta) + rnorm(n)
    fit <- glmnet(X, y, lambda = lambda_grid)
    ea[i, ] <- (as.vector(predict(fit, newx = matrix(xa, nrow = 1))) - truth_a)^2
    eb[i, ] <- (as.vector(predict(fit, newx = matrix(xb, nrow = 1))) - truth_b)^2
  }
  la <- lambda_grid[which.min(colMeans(ea))]
  lb <- lambda_grid[which.min(colMeans(eb))]
  c(lambda_a = la, lambda_b = lb, ratio = lb / la)
}

seed_tab <- t(sapply(c(1, 7, 100, 432, 2024), argmin_for_seed))
rownames(seed_tab) <- paste("seed", c(1, 7, 100, 432, 2024))
round(seed_tab, 5)
```

|           | lambda_a | lambda_b | ratio   |
|-----------|----------|----------|---------|
| seed 1    | 0.02872  | 0.10026  | 3.49034 |
| seed 7    | 0.02237  | 0.12246  | 5.47395 |
| seed 100  | 0.02128  | 0.14227  | 6.68589 |
| seed 432  | 0.01832  | 0.17377  | 9.48774 |
| seed 2024 | 0.03175  | 0.13534  | 4.26311 |

The optimal  $\lambda$  for  $\mathbf{x}_a$  moves over roughly 0.018–0.032 and for  $\mathbf{x}_b$  over 0.100–0.174, so the *ratio* ranges from about 3.5 to 9.5 depending only on the seed.

The conclusion that survives is therefore the *ordering* and its mechanism —  $\lambda_b^* > \lambda_a^*$  by a substantial factor, for every seed tried — and not the specific multiple. I report 9.5 because that is what this run gives, but it should be read as “several times larger”, not as an estimate of a fixed quantity.

### Why the best $\lambda$ for $\mathbf{x}_b$ is larger

Because the design is orthogonal in expectation ( $X_j$  are independent standard normals), the Lasso at  $\mathbf{x}$  is close to a soft-threshold of the least-squares coefficient, and the error at a target point that isolates coordinate  $j$  decomposes as

$$\mathbb{E}(\hat{\beta}_0 + \hat{\beta}_j - \beta_j)^2 = \underbrace{(\mathbb{E}[\hat{\beta}_0 + \hat{\beta}_j] - \beta_j)^2}_{\text{squared bias}} + \underbrace{\text{Var}(\hat{\beta}_0 + \hat{\beta}_j)}_{\text{variance}}.$$

Shrinkage buys a variance reduction at the price of a bias that grows with  $\lambda$ , and the size of that price is set entirely by how large  $\beta_j$  is.

- For  $\mathbf{x}_a$ ,  $\beta_2 = 0.2737$  is a real effect. Shrinking it costs bias immediately, and at the top of the grid the coefficient is driven to zero and the prediction collapses to  $\bar{y} \approx 0$ , leaving a squared bias of about  $0.2737^2 = 0.0749$ . Adding  $\text{Var}(\bar{y}) = (\|\beta\|^2 + 1)/n = 1.3614/100 = 0.0136$  predicts an error of 0.0885 at  $\lambda \rightarrow \infty$ , which matches the observed 0.0904. The variance that shrinkage could have saved is far smaller than that, so the optimum sits at the low end of the grid.
- For  $\mathbf{x}_b$ ,  $\beta_{10} = 0.0552$  is statistically indistinguishable from zero at  $n = 100$ . Setting  $\hat{\beta}_{10} = 0$  costs a squared bias of only  $0.0552^2 = 0.0030$  while removing *all* of the estimation variance in that coordinate. The same calculation gives  $0.0030 + 0.0136 = 0.0166$  at  $\lambda \rightarrow \infty$ , against the observed 0.0177 — and against a minimum of 0.0167. In other words, for  $\mathbf{x}_b$  the “shrink it to nothing” solution is within noise of optimal, which is exactly why the curve is flat on the right and why the minimizing  $\lambda$  is large.

The general statement is that the optimal amount of shrinkage at a target point depends on the target point, not just on the model: it is governed by the signal-to-noise ratio of the particular coefficients that the target loads on. A single  $\lambda$  chosen by cross-validation optimizes the *average* over the test distribution, and so is necessarily too large for targets loading on strong coefficients and too small for targets loading on weak ones. Here the two targets sit at opposite ends of the coefficient decay  $0.4\sqrt{j}$ , which is what makes the discrepancy so visible.

---

## Question 3 [25 pts] (Shrinkage methods and testing error)

### Original question

Predict the number of applications (**Apps**) received using the other variables in the **College** data from ISLR2.

- [5 pts] Split with the code given below. Fit a linear model with all inputs using `lm()` and report the test MSE.
- [5 pts] Compare Lasso and ridge, training by cross-validation on the training set. Report the test error for both using `lambda.min` and `lambda.1se`.
- [15 pts] Read the `glmnet` documentation on **relaxed** fits and the tuning parameter **gamma**, and summarize the idea in terms of the loss function, written in terms of **y** and **X**. Then implement the feature, use cross-validation to find the optimal  $\lambda$  and  $\gamma$ , and report the test error of the optimal model.

### (a) Least squares

```
data(College)

set.seed(7)
test_idx <- sample(nrow(College), 177)
train <- College[-test_idx, ]
test <- College[test_idx, ]

fit_lm <- lm(Apps ~ ., data = train)
mse_lm <- mean((predict(fit_lm, newdata = test) - test$Apps)^2)

cat("training / test sizes:", nrow(train), "/", nrow(test), "\n")
cat("least squares test MSE:", round(mse_lm, 2), "\n")
```

```
training / test sizes: 600 / 177
least squares test MSE: 961142.3
```

The full least-squares model gives a test MSE of 961,142. **Apps** runs into the tens of thousands, so the absolute scale is large and only the comparisons between methods are meaningful. Note that  $n = 600$  and  $p = 17$  here, so this is a low-dimensional problem and least squares is already a reasonable estimator — we should not expect dramatic gains from shrinkage.

### (b) Lasso and ridge

`glmnet` takes a numeric design matrix, so **Private** (a two-level factor) is expanded by `model.matrix()`, and the intercept column it produces is dropped because `glmnet` fits its own.

```
xtrain <- model.matrix(Apps ~ ., data = train)[, -1]
xtest <- model.matrix(Apps ~ ., data = test)[, -1]
ytrain <- train$Apps
ytest <- test$Apps

set.seed(7)
cv_lasso <- cv.glmnet(xtrain, ytrain, alpha = 1)
set.seed(7)
cv_ridge <- cv.glmnet(xtrain, ytrain, alpha = 0)
```

```

test_error <- function(cv, s) mean((predict(cv, newx = xtest, s = s) - ytest)^2)
nonzero    <- function(cv, s) sum(as.vector(coef(cv, s = s))[-1] != 0)

cat(sprintf("lasso lambda.min = %8.3f  test MSE = %10.1f  nonzero = %2d\n",
            cv_lasso$lambda.min, test_error(cv_lasso, "lambda.min"), nonzero(cv_lasso, "lambda.min")))
cat(sprintf("lasso lambda.1se = %8.3f  test MSE = %10.1f  nonzero = %2d\n",
            cv_lasso$lambda.1se, test_error(cv_lasso, "lambda.1se"), nonzero(cv_lasso, "lambda.1se")))
cat(sprintf("ridge lambda.min = %8.3f  test MSE = %10.1f\n",
            cv_lasso$lambda.min, test_error(cv_lasso, "lambda.min")))
cat(sprintf("ridge lambda.1se = %8.3f  test MSE = %10.1f\n",
            cv_lasso$lambda.1se, test_error(cv_lasso, "lambda.1se")))

sel <- rownames(coef(cv_lasso, s = "lambda.1se"))[-1][as.vector(coef(cv_lasso, s = "lambda.1se"))[-1] != 0]
cat("lasso lambda.1se keeps:", paste(sel, collapse = ", "), "\n")

```

```

lasso lambda.min =    2.135  test MSE =   946414.7  nonzero = 16
lasso lambda.1se =  390.815  test MSE =  1076205.5  nonzero =  3
ridge lambda.min =   364.476  test MSE =   875548.9
ridge lambda.1se = 1772.301  test MSE =  1338039.9
lasso lambda.1se keeps: Accept, Top10perc, Expend

```

| method        | rule       | $\lambda$ | test MSE       | nonzero |
|---------------|------------|-----------|----------------|---------|
| least squares | —          | —         | 961,142        | 17      |
| Lasso         | lambda.min | 2.13      | 946,415        | 16      |
| Lasso         | lambda.1se | 390.82    | 1,076,206      | 3       |
| ridge         | lambda.min | 364.48    | <b>875,549</b> | 17      |
| ridge         | lambda.1se | 1772.30   | 1,338,040      | 17      |

Three observations.

First, **lambda.min beats lambda.1se for both methods, by a lot.** This is the expected ordering when  $n \gg p$ : the one-standard-error rule deliberately overshrinks in exchange for a simpler model, and here there is no high-dimensional variance problem for that extra simplicity to solve. The **lambda.1se** Lasso keeps only **Accept, Top10perc** and **Expend** and pays about 13% in test MSE for the parsimony.

Second, **ridge at lambda.min is the best model tested**, at 875,549, roughly 9% better than least squares. The **College** predictors are strongly correlated with one another (**Apps, Accept** and **Enroll** in particular move together), so a modest uniform shrinkage stabilizes the coefficients without discarding information. This is the opposite regime from Question 1: the signal here is dense rather than sparse, which is the situation in which ridge is supposed to win.

Third, **Lasso at lambda.min barely shrinks at all** ( $\lambda = 2.13$ , 16 of 17 variables retained) and lands essentially on top of least squares. With  $p = 17$  and  $n = 600$  there is very little to be gained from selection.

### (c) Relaxed fits and the gamma parameter

**The idea in terms of the loss function.** Let  $\hat{\beta}^L(\lambda)$  be the usual Lasso solution,

$$(\hat{\beta}_0^L, \hat{\beta}^L(\lambda)) = \arg \min_{\beta_0, \beta} \frac{1}{2n} \|\mathbf{y} - \beta_0 \mathbf{1} - \mathbf{X}\beta\|_2^2 + \lambda \|\beta\|_1,$$

and let

$$\mathcal{A}_\lambda = \{j : \hat{\beta}_j^L(\lambda) \neq 0\}$$

be its active set. The **relaxed fit** at  $\lambda$  is the *unpenalized* least-squares fit restricted to that active set — the same loss, with the penalty removed and the excluded coefficients held at zero:

$$(\tilde{\beta}_0, \tilde{\beta}(\lambda)) = \arg \min_{\beta_0, \beta : \beta_j = 0 \ \forall j \notin \mathcal{A}_\lambda} \frac{1}{2n} \|\mathbf{y} - \beta_0 \mathbf{1} - \mathbf{X}\beta\|_2^2.$$

Equivalently  $\tilde{\beta}_{\mathcal{A}_\lambda} = (\mathbf{X}_{\mathcal{A}_\lambda}^\top \mathbf{X}_{\mathcal{A}_\lambda})^{-1} \mathbf{X}_{\mathcal{A}_\lambda}^\top \mathbf{y}$  after centering, where  $\mathbf{X}_{\mathcal{A}_\lambda}$  collects the active columns. The parameter  $\gamma \in [0, 1]$  then interpolates between the two linear predictors,

$$\tilde{\eta}_{\lambda, \gamma}(\mathbf{x}) = (1 - \gamma) \tilde{\eta}_\lambda(\mathbf{x}) + \gamma \hat{\eta}_\lambda(\mathbf{x}), \quad \text{equivalently} \quad \tilde{\beta}(\lambda, \gamma) = (1 - \gamma) \tilde{\beta}(\lambda) + \gamma \hat{\beta}^L(\lambda),$$

so that  $\gamma = 1$  recovers the ordinary Lasso and  $\gamma = 0$  gives the fully debiased least-squares-on-the-active-set fit. Because  $\tilde{\beta}$  and  $\hat{\beta}^L$  are both supported on  $\mathcal{A}_\lambda$ , so is every blend:  $\gamma$  changes *how much shrinkage is applied inside the selected set* without changing the selected set itself.

The point of the construction is that the Lasso uses a single  $\lambda$  for two jobs — deciding which variables enter, and deciding how much to shrink the ones that do. A  $\lambda$  large enough to produce a clean, small active set also shrinks the surviving coefficients far more than prediction accuracy wants. Relaxed fitting splits the two jobs:  $\lambda$  selects,  $\gamma$  shrinks, and cross-validation tunes them jointly. This is why relaxed fits typically select *sparser* models than plain Lasso — once the bias can be undone by  $\gamma$ , cross-validation is free to prefer a larger  $\lambda$ .

I checked the endpoints of this definition directly against the package: `coef(fit, gamma = 1)` reproduces the plain Lasso path exactly, `coef(fit, gamma = 0)` matches `lm()` refit on the active set, and `gamma = 0.5` is the exact midpoint of the two coefficient vectors. Note that the blend is defined on the linear predictor (equivalently on the coefficients), not as a mixture of two loss functions: a mixture of losses would add a term not involving  $\beta$ , which is an additive constant and would collapse  $\gamma$  into a rescaling of  $\lambda$ .

### Implementation.

```
set.seed(7)
cv_relax <- cv.glmnet(xtrain, ytrain, alpha = 1, relax = TRUE)

cat("gamma grid:", paste(cv_relax$relaxed$gamma, collapse = ", "), "\n\n")

cat(sprintf("relaxed lambda.min = %8.3f   gamma.min = %.2f   test MSE = %10.1f   nonzero = %2d\n",
  cv_relax$relaxed$lambda.min, cv_relax$relaxed$gamma.min,
  mean((predict(cv_relax, newx = xtest, s = "lambda.min", gamma = "gamma.min") - ytest)^2),
  sum(as.vector(coef(cv_relax, s = "lambda.min", gamma = "gamma.min"))[-1] != 0)))

cat(sprintf("relaxed lambda.1se = %8.3f   gamma.1se = %.2f   test MSE = %10.1f\n",
  cv_relax$relaxed$lambda.1se, cv_relax$relaxed$gamma.1se,
  mean((predict(cv_relax, newx = xtest, s = "lambda.1se", gamma = "gamma.1se") - ytest)^2)))
```

gamma grid: 0, 0.25, 0.5, 0.75, 1

|                      |         |             |      |            |           |           |    |
|----------------------|---------|-------------|------|------------|-----------|-----------|----|
| relaxed lambda.min = | 2.135   | gamma.min = | 1.00 | test MSE = | 946414.7  | nonzero = | 16 |
| relaxed lambda.1se = | 749.549 | gamma.1se = | 0.00 | test MSE = | 1040187.9 |           |    |

Cross-validation searches the default grid  $\gamma \in \{0, 0.25, 0.5, 0.75, 1\}$  and selects  $\hat{\lambda} = 2.13$  with  $\hat{\gamma} = 1$ , giving a test MSE of **946,415** with 16 nonzero coefficients.

$\hat{\gamma} = 1$  means **cross-validation chose the plain Lasso and declined the relaxation entirely**, and the test MSE is identical to the `lambda.min` Lasso in part (b), as it must be. That is a real result rather than a bug, and it is what the setting predicts: the selected  $\lambda = 2.13$  is so small that the Lasso is barely shrinking anything, so there is essentially no bias for  $\gamma$  to remove — and debiasing a 16-variable active set on 600 observations adds variance without a compensating gain. Relaxed fitting pays off when  $\lambda$  has to be large, which happens when  $p$  is large relative to  $n$ ; with  $p = 17$  and  $n = 600$  that pressure is absent.

The one-standard-error rule tells the complementary half of the story: there  $\hat{\lambda}_{1se} = 749.55$  with  $\hat{\gamma}_{1se} = 0$ , i.e. a much heavier penalty for selection combined with *complete* debiasing, and it achieves 1,040,188 against the plain Lasso `lambda.1se` value of 1,076,206. That is exactly the mechanism described above — a larger  $\lambda$  becomes affordable once  $\gamma$  can undo its bias — but it is still worse than simply not overshrinking in the first place.

For this data set, then, the best model remains **ridge at lambda.min, with a test MSE of 875,549**.

---

## Question 4 [10 pts] (Penalized logistic regression)

### Original question

The `golub` data in the `multtest` package contain 3051 genes from 38 tumor mRNA samples, with outcome `golub.cl` indicating Acute Lymphoblastic Leukemia (ALL) or Acute Myeloid Leukemia (AML). Gene expressions are highly correlated, so consider the elastic net for both sparsity and correlation. Fit logistic regression to this data set, use a grid of  $\alpha$  values in  $[0, 1]$ , and report the best  $\alpha$  and  $\lambda$  using 19-fold cross-validation.

### Setup

`golub` is stored genes-by-samples, so it must be transposed to the  $38 \times 3051$  design `glmnet` expects. Two choices matter for the comparison to be meaningful:

- **The folds are fixed across  $\alpha$ .** A single `foldid` vector is generated once and reused for every  $\alpha$ , so that differences in the cross-validated deviance reflect the penalty and not the fold partition. With  $n = 38$  and 19 folds, each fold holds out exactly 2 observations, so this is essentially leave-two-out and the fold assignment would otherwise be a large source of noise.
- **The criterion is binomial deviance**, the natural likelihood-based measure. Misclassification error is too coarse here: with 38 samples it takes only integer values in steps of  $1/38$  and ties across large stretches of the grid.

```
library(multtest)
data(golub)

X <- t(golub)           # 38 samples x 3051 genes
Y <- golub.cl           # 0 = ALL, 1 = AML

set.seed(7)
foldid <- sample(rep(1:19, length.out = length(Y)))
```

```

alphas <- seq(0, 1, by = 0.05)
tab <- do.call(rbind, lapply(alphas, function(a) {
  cv <- cv.glmnet(X, Y, family = "binomial", alpha = a,
                  foldid = foldid, type.measure = "deviance")
  i <- which(cv$lambda == cv$lambda.min)
  data.frame(alpha = a, lambda.min = cv$lambda.min,
             cv.deviance = cv$cvm[i], nonzero = cv$nonzero[i])
}))

```

```

cat("dimensions:", dim(X), "    class counts:", paste(table(Y), collapse = " / "), "\n")

```

```

dimensions: 38 3051    class counts: 27 / 11

```

```

knitr::kable(tab, digits = 5, row.names = FALSE,
             caption = "19-fold CV over the alpha grid, folds held fixed.")

```

Table 2: 19-fold CV over the alpha grid, folds held fixed.

| alpha | lambda.min | cv.deviance | nonzero |
|-------|------------|-------------|---------|
| 0.00  | 3.91451    | 0.26403     | 3051    |
| 0.05  | 0.07829    | 0.14486     | 507     |
| 0.10  | 0.03915    | 0.14717     | 329     |
| 0.15  | 0.02610    | 0.14625     | 245     |
| 0.20  | 0.01957    | 0.14515     | 192     |
| 0.25  | 0.01566    | 0.14477     | 158     |
| 0.30  | 0.01305    | 0.14605     | 133     |
| 0.35  | 0.01118    | 0.14880     | 113     |
| 0.40  | 0.00979    | 0.15291     | 99      |
| 0.45  | 0.00870    | 0.15684     | 91      |
| 0.50  | 0.00783    | 0.16208     | 86      |
| 0.55  | 0.00712    | 0.16809     | 79      |
| 0.60  | 0.00652    | 0.17705     | 69      |
| 0.65  | 0.00602    | 0.18726     | 56      |
| 0.70  | 0.00559    | 0.19745     | 52      |
| 0.75  | 0.00522    | 0.20808     | 49      |
| 0.80  | 0.00489    | 0.21889     | 45      |
| 0.85  | 0.00461    | 0.21937     | 35      |
| 0.90  | 0.00435    | 0.21033     | 29      |
| 0.95  | 0.00412    | 0.18461     | 22      |
| 1.00  | 0.00391    | 0.15432     | 14      |

```

best <- tab[which.min(tab$cv.deviance), ]

```

## Result

The best model is  $\hat{\alpha} = 0.25$  with  $\hat{\lambda} = 0.01566$ , achieving a cross-validated binomial deviance of 0.1448 with 158 genes retained.

Three points about this answer.

First, **the elastic net does beat both of its endpoints, but the optimum is very flat.** Everything in  $\alpha \in [0.05, 0.30]$  lands between 0.1448 and 0.1472, a spread far smaller than the fold-to-fold variability one should expect from 19 folds of 2 observations. The honest reading is that the data support “a small but nonzero amount of  $L_1$ ” rather than “ $\alpha = 0.25$  specifically” — with a different fold the argmin moves around inside that plateau. The *shape* of the curve is the reliable finding; the argmin is not.

Second, **pure ridge ( $\alpha = 0$ ) is clearly the worst choice**, at 0.2640, nearly twice the deviance of the optimum. With  $p = 3051$  and  $n = 38$ , keeping all 3051 genes is untenable no matter how hard they are shrunk; some sparsity is essential.

Third, **pure Lasso ( $\alpha = 1$ ) is competitive but not optimal** (0.1543 with only 14 genes). This is exactly the behaviour the question’s preamble anticipates: because the gene expressions are highly correlated, the Lasso picks one gene out of each correlated cluster and discards the rest, which is unstable and throws away replicated evidence. The small ridge component in the elastic net restores the grouping effect, letting whole correlated clusters enter together — note that the chosen model keeps 158 genes rather than 14 — and that averaging over correlated measurements is what buys the improvement in deviance.

A caveat worth stating plainly: with  $n = 38$  and two classes split 27/11, every number in this table carries substantial uncertainty, and the cross-validated deviance is being used both to choose  $(\alpha, \lambda)$  and to report performance. The value 0.1448 should not be read as an unbiased estimate of this procedure’s out-of-sample deviance.